RoBO-Benchmark: Probleme und Änderungsvorschläge

Sortiert von wichtig nach unwichtig. **Methodisch = Ja**: Das Problem beeinflusst die wissenschaftliche Aussage der Ergebnisse. **Nein**: Technik, Wartbarkeit oder Ordnung. Die Punkte 1–4 (markiert) haben den größten Einfluss; solange sie offen sind, ist der Vergleich GP_BO gegen PROBO nicht fair.

#	Problem	Ort	Änderungsvorschlag	Methodisch
1	PROBO nutzt eine feste Längenskala 1.0 auf dem Rohraum $[-5,5]^d$. Die Punkte sind praktisch unkorreliert, PROBO sucht faktisch zufällig. GP_BO normiert und fittet dagegen.	`probo.py:181`	Eingaben auf $[0,1]^d$ normieren und die Längenskala per Marginal Likelihood schätzen (z. B. aus dem GP_BO-Fit übernehmen), mindestens aber mit $\sqrt{d}$ skalieren.	**Ja**
2	PROBO hat kein Rauschmodell (Jitter 1e-8) und interpoliert bei *stochastic_input* das Rauschen. GP_BO lernt es mit.	`uncertainty_aware_bo.py:171`	Rauschterm $\sigma^2 \cdot I$ schätzen oder aus dem GP_BO-Fit übernehmen.	**Ja**
3	x_rec ist der beste *verrauschte* Wert. Bei *stochastic_input* gewinnen Punkte mit zufällig günstigem Rauschen.	`experiment_runner.py:205`	Nach dem Posterior-Mittelwert empfehlen (argmax μ über die Trainingspunkte); mu_train wird bereits gespeichert. Die Regel in der Arbeit begründen.	**Ja**
4	Off-by-one: x_rec wird vor dem Anhängen von next_x gewählt. Die letzte der 100 Auswertungen kann nie empfohlen werden, M1–M6 hängen eine Iteration hinterher.	`experiment_runner.py:205`	x_rec und die Metriken erst nach dem Anhängen von next_x berechnen.	**Ja**
5	Der Term mu_bounds_diff liegt bei c=50 grob bei 100/n und dominiert vermutlich die Akquisition. PROBO exploriert dann fast nur.	`uncertainty_aware_bo.py:245`	Größe der Terme loggen und vergleichen. c und rho am Paper kalibrieren oder eine Sensitivitätsanalyse machen.	**Ja**
6	Beim Prior-Fehler wird die Richtung zufällig gezogen (direction="random"). Optimistische und pessimistische Fehler werden vermischt und mitteln sich bei 10 Wiederholungen teilweise heraus.	`prior_error.py:40`	Beide Richtungen als getrennte Stufen rechnen (eigene Spalte *direction* in der task_id).	**Ja**
7	PROBO nutzt in UCB/GLCB die Varianz statt der Standardabweichung. Das weicht von Definition 11 im Paper ab.	`uncertainty_aware_bo.py:223`	Auf $\sqrt{\text{Var}}$ umstellen oder die Abweichung in der Arbeit begründen (bisher nur im Code dokumentiert).	**Ja**
8	Die Metriken schneiden $x + \xi$ am Rand des Suchraums ab. Das verzerrt die Rauschverteilung nahe am Rand.	`metrics.py:355`	Nicht abschneiden, da BBOB auch außerhalb des Suchraums definiert ist, oder das Abschneiden bewusst begründen.	**Ja**
9	Die Metrik-Parameter sind fest (z. B. $\sigma = 0.25$), die Szenario-Intensität geht aber bis 0.5. Das ist bewusst so gewählt, aber noch nicht erklärt.	`config.py`	In der Arbeit begründen. Optional zusätzlich die Metriken mit σ = Szenario-Intensität berechnen.	**Ja**
10	iter_time_s enthält auch die Metrikberechnung. Ein Laufzeitvergleich der Verfahren ist damit verfälscht.	`experiment_runner.py:182`	Die Zeit für Fit und Akquisition getrennt messen (algo_time_s).	**Ja***
11	Die Config stimmt nicht mit dem Code überein: GP_BO steht auf „Matern52“, genutzt wird RBF. prior_set_width bei PROBO wird nirgends verwendet.	`config.py`	Config an den Code anpassen oder die Werte im Code tatsächlich verwenden.	Nein**

#	Problem	Ort	Änderungsvorschlag	Methodisch
12	requirements.txt hat keine Versionen. Die Ergebnisse sind nicht exakt reproduzierbar.	`requirements.txt`	Versionen mit pip freeze festlegen und im Anhang der Arbeit nennen.	Nein
13	Die task_id enthält weder Budget noch Hyperparameter. Nach einer Config-Änderung werden alte Ergebnisse beim Fortsetzen stillschweigend übernommen.	`experiment_design.py:50`	Einen Hash der Config in die task_id/CSV aufnehmen oder eine neue Ausgabedatei erzwingen.	Nein
14	Fehlgeschlagene Experimente bleiben als error-Zeilen in der CSV, auch nach erfolgreicher Neuberechnung.	`experiment_runner.py:259`	Im Postprocessing nach status == "ok" filtern oder error-Zeilen beim Neustart entfernen.	Nein
15	qmc.LatinHypercube(seed=rng) ist in neuen SciPy-Versionen veraltet.	`probo.py:168`	Wie in nominal.py den Helfer _latin_hypercube mit rng= verwenden.	Nein
16	Die f_opt-Abfrage lässt einen Ordner exdata/ im Arbeitsverzeichnis zurück.	`coco_setup.py:41`	Temporären Ordner (tempfile) nutzen und danach komplett löschen.	Nein
17	make_benchmark negiert f_opt und widerspricht damit coco_setup. Die Funktion ist toter Code.	`metrics.py:313`	Entfernen.	Nein
18	Altlasten: relevance_pursuit.py.py ist doppelt vorhanden, fünf Dateien sind leer, die Szenarien adversarial/shift sind nur Platzhalter.	mehrere	Löschen oder als TODO kennzeichnen.	Nein

* sobald die Laufzeit ausgewertet wird. ** kein methodischer Fehler im Code, aber riskant für die Beschreibung des Setups in der Arbeit.